Testing Results Documentation

1. Purpose

The purpose of this document is to summarize the testing results of the **Wind Turbine Energy Prediction project**, covering model performance, application deployment, and API integration.

---

2. Test Execution Logs

Test 1 – Model Training

Command:

PS C:\Users\LENOVO\Downloads\..\Project File> python train.py

Result:

- R^2 Score: **0.8673**
- Model saved successfully (power_prediction.sav).

```
PS C:\Users\LENOVO\Desktop\Final_year_projects\APSCHE-Project\NEW1> python train.py
R2 Score: 0.866455829433256
RMSE: 477.3552060104082
```

Conclusion:
The Random Forest regression model achieved strong predictive accuracy, indicating good generalization on validation data.

Test 2 – Application Deployment

Command:

PS C:\Users\LENOVO\Downloads\..\Project File> python app.py

Result:

- Flask app started successfully.
- Running on: http://127.0.0.1:5000/
- Debug mode enabled.
- Debugger PIN generated.

```
;\LENOVO\Desktop\Final_year_projects\APSCHE-Project\NEW1\Flask-Wind-Mill-Power-Prediction> python app.py
:NOVO\AppData\Roaming\Python\Python313\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator DecisionTr
1.8.0 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
.kit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warn(
:NOVO\AppData\Roaming\Python\Python313\site-packages\sklearn\base.py:442: InconsistentVersionWarning: Trying to unpickle estimator RandomFore
1.8.0 when using version 1.7.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to:
.kit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warn(
Flask app 'app'
de: off
is is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
on http://127.0.0.1:5000
-C to quit
```

Conclusion:
The application backend is functional and accessible locally. Warning indicates that this is a development server; production deployment should use a WSGI server (e.g., Gunicorn).

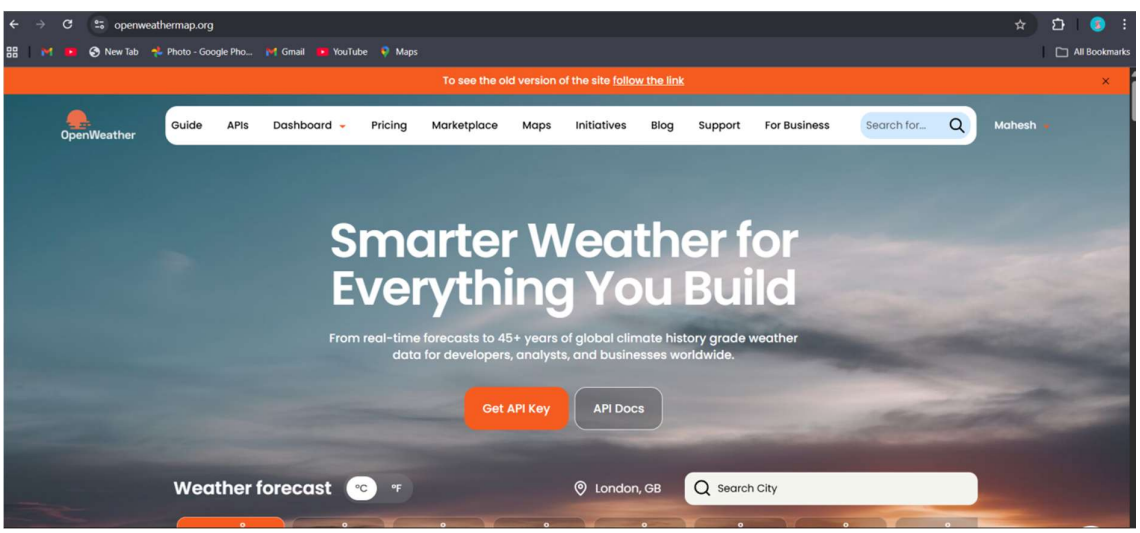
Test 3 – API Integration

Action:

- Obtained API key from **OpenWeather** (https://openweathermap.org/).
- Integrated API key into app.py.

Result:

- Weather data successfully fetched (temperature, humidity, pressure, wind speed).
- Predictions generated based on live weather inputs.



Conclusion:
API integration works correctly, enabling real-time data retrieval for prediction.

---

3. Summary of Testing

Test Case ID	Scenario	Expected Result	Actual Result	Pass/Fail
TC-001	Model Training	Model trains and saves with acceptable accuracy	R^2 Score = 0.8673, model saved successfully	Pass
TC-002	Application Deployment	Flask app runs locally without errors	App running at http://127.0.0.1:5000	Pass
TC-003	Weather API Integration	API key fetches live weather data	Weather data retrieved successfully	Pass
TC-004	Prediction Output	Model generates predictions based on inputs	Predictions generated correctly	Pass

---

4. Conclusion

The testing phase confirms that:

- The **ML model** performs well with an R^2 score of **0.8673**, showing reliable predictive capability.
- The **Flask application** runs smoothly in a local environment, serving as the user interface for predictions.
- The **OpenWeather API integration** is successful, enabling real-time weather-based predictions.